

JOURDIN

ROUCHOUZE

Pour le 19/01/2025

Clem

Hélène

INFO1 TP2

Projet Démineur : Rapport

ENSSAT

L A N N I O N

Table des matières

Introduction	Page 3
I. Préliminaires	Page 4
I.1. Interface graphique	Page 4
I.2. Mode de fonctionnement	Page 5
II. Modélisation	Page 6
II.1 Conception du jeu	Page 6
II.2 Intelligence Artificielle	Page 7
III. Stratégie	Page 9
III.1 Conception du jeu	Page 9
III.2 Intelligence Artificielle	Page 10
IV. Pour aller plus loin	Page 12
IV.1 Problèmes repérés	Page 12
IV.2 Pistes d'amélioration	Page 14
Conclusion	Page 15
Sources	Page 16

Introduction

Ce rapport décrit les moyens algorithmiques employés dans le but de concevoir à la fois le jeu du démineur dans son ensemble et une Intelligence Artificielle capable d'aider le joueur dans sa prise de décision. La principale contrainte imposée est l'implémentation de l'Intelligence Artificielle à l'aide d'un arbre de décision permettant d'informer le joueur sur la sûreté d'une case.

Le jeu consiste en une grille de taille variable où se cachent des mines, le joueur ne doit pas révéler de cases où se trouve une mine. Pour l'aider, sur une case révélée est indiqué le nombre de mines adjacentes à cette case.

Le jeu Démineur est adaptatif quant à la taille de la grille de jeu et au nombre de bombes sur le plateau.

Nous verrons dans un premier temps quelques préliminaires essentiels à la compréhension de l'algorithme et de la mise en place du jeu. Puis nous étudierons la modélisation utilisée pour la conception du jeu. Viendra ensuite l'explication de la stratégie employée. Pour finir, nous verrons ce qui peut être amélioré.

Prérequis : Pour avoir accès à l'interface graphique, il faut installer la bibliothèque sdl2 via la commande `$ sudo apt install libsdl2-dev`, dans le terminal linux.

I. Préliminaires

I.1. Interface graphique

Afin de faciliter la prise en main du jeu du démineur, nous avons utilisé la bibliothèque SDL2 en C pour créer une interface graphique.

Le but est de créer une fenêtre dans laquelle seront affichées les cases de jeu (le fonctionnement du jeu étant géré en arrière plan à l'aide de matrices).

Pour cela, il faut d'abord initialiser SDL avec *SDL_Init* puis créer la fenêtre d'affichage et le renderer (qui permet de gérer l'affichage dans la fenêtre) avec *SDL_CreateWindow* et *SDL_CreateRenderer*.

On peut ensuite, avec *init_jeu*, afficher des carrés gris qui représentent les cases non dévoilées par le joueur.

Dans la fonction principale du jeu, on crée une boucle qui va continuer d'afficher la fenêtre de jeu tant que la partie n'est pas finie ou tant que le joueur n'a pas fermé de lui-même la fenêtre de jeu.

Pour cela, à chaque tour de boucle, on teste si le joueur clique sur la croix de la fenêtre ou s'il clique sur une case dans la fenêtre de jeu. Si le joueur clique dans la fenêtre, on regarde quel clic a été effectué et où dans la fenêtre avec *SDL_BUTTON* et *SDL_GetMouseState*, et on joue en conséquence. On peut ensuite appeler la fonction *mettre_a_jour_affichage* qui va dessiner dans la case demandée le symbole demandé (un chiffre ou un drapeau selon l'action réalisée).

Toutes les fonctions *afficher_chiffre* dessinent des rectangles dans la case demandée de l'épaisseur donnée par la variable *EPAIS* définie au début du programme. *afficher_bombe* ou *afficher_drapeau* dessinent toujours les mêmes symboles, quelque soit l'épaisseur donnée.

I.2. Mode de fonctionnement

Pour ce projet, nous reprenons le mode de fonctionnement classique du jeu démineur avec le clic droit pour poser ou enlever un drapeau et le clic gauche pour casser une case. Nous avons également repris la fonctionnalité du jeu original permettant de dévoiler toutes les cases adjacentes à un numéro lorsque ce numéro a déjà le bon nombre de drapeaux dans ses alentours.

Une fonctionnalité que nous avons implementée mais qui n'existe pas dans le démineur classique est l'intervention de l'Intelligence Artificielle. En effet, lorsque l'on effectue un clic molette sur une case non dévoilée du plateau, le terminal affiche les prévisions de l'Intelligence Artificielle pour cette case (fonctionnement de l'Intelligence Artificielle détaillé dans les parties II.2 et III.2).

Une dernière différence entre le jeu original et notre version est que le jeu original est fait de telle sorte qu'il soit impossible de perdre dès le premier coup car le plateau est construit après le premier clic du joueur, de manière à ce que ce premier clic soit au niveau d'un 0. Nous ne prenons pas en compte cette fonctionnalité, il est donc possible de perdre dès le premier coup dans notre version.

L'interface graphique (détaillé en partie I.1) permet donc d'afficher l'état courant du plateau et d'effectuer les 4 actions possibles : révéler une case, placer un drapeau, retirer un drapeau et utiliser l'Intelligence Artificielle.

Le terminal affiche le temps écoulé jusqu'alors (mise à jour à chaque action effectuée), le nombre de mines restantes ainsi que l'état courant du plateau (état normalement déjà affiché par l'affichage graphique).

4 paramètres peuvent être modifiés au début du fichier demineur.c :

```
9
10  #define MINES 40
11  #define GRILLE 16
12  #define GRIS 189
13  #define EPAIS 3
14
```

- MINES correspond au nombre de mines présentes sur le plateau.
- GRILLE correspond à la taille du plateau (matrice de taille GRILLE*GRILLE).
- GRIS correspond à la couleur du fond des cases (en RGB).
- EPAIS est un paramètre définissant l'épaisseur des chiffres affichés par l'interface graphique.

II. Modélisation

II.1. Conception du jeu

Nom	Type	Valeurs possibles	Description
plateau	Matrice de char de taille GRILLE* GRILLE	• 'M' : Mine • '0' – '8' : Nombre de mines adjacentes	Matrice contenant l'état du plateau, indépendamment de l'état d'avancée du joueur dans la découverte des cases. La matrice <i>plateau</i> sauvegarde simplement les emplacements des mines et des chiffres qui en découlent. Elle n'évolue pas au cours du jeu.
visible	Matrice de char de taille GRILLE* GRILLE	• 'D' : Drapeau • '#' : Case non découverte • 'X' : Mine (dévoilée que si défaite) • '0' – '8' : Nombre de mines adjacentes	Matrice rendant compte de l'état d'avancée du joueur dans la partie. Elle sauvegarde la donnée des cases déjà découvertes (et ce qu'il y a en dessous, récupéré à partir de la matrice <i>plateau</i>) et des drapeaux placés. Elle est mise à jour après chaque action du joueur (sauf le recours à l'Intelligence Artificielle).

II.2. Intelligence Artificielle

(x_depart, y_depart) : coordonnées de la case choisie pour être étudiée par l'Intelligence Artificielle

$$\text{TAILLE_ELT} = (2 * \text{MILIEU}) + 1$$

Nom	Type	Valeurs possibles	Description
ELT	Type correspondant à une matrice de char de taille TAILLE_ELT*TAILLE_ELT	Théoriquement tout caractère	Type générique utilisé comme valeur dans les nœuds des arbres manipulés ici.
ARBRE	Type correspondant à un arbre.	Trois attributs : • val : valeur de type ELT • g : fils gauche (de type ARBRE) • d : fils droit (de type ARBRE)	Type arbre. Les arbres manipulés ici sont des arbres binaires stricts.
alentours	ELT	• 'D' : Drapeau • '#' : Case non découverte • '-' : Hors-plateau • '0' – '8' : Nombre de mines adjacentes	Matrice affichant les cases aux alentours de la case (x_depart, y_depart) dans un rayon de MILIEU cases.
mat_mine	ELT	• 'D' : Drapeau • '#' : Case non découverte • '-' : Hors-plateau • '0' – '8' : Nombre de mines adjacentes	Copie de la matrice <i>alentours</i> , avec un 'D' en son centre (correspondant à la case (x_depart, y_depart)).
mat_pas_mine	ELT	• 'D' : Drapeau • '#' : Case non découverte • '-' : Hors-plateau • '0' – '8' : Nombre de mines adjacentes	Copie de la matrice <i>alentours</i> , avec un '#' en son centre (correspondant à la case (x_depart, y_depart)). <u>Remarque</u> : cela correspond donc exactement à la matrice <i>alentours</i> .
a_mine	ARBRE	Cf ARBRE	ARBRE ayant pour racine <i>mat_mine</i> et pour enfants toutes les combinaisons de possibilités d'état ('#' ou 'D') pour les cases non découvertes dans <i>alentours</i> .
a_pas_mine	ARBRE	Cf ARBRE	ARBRE ayant pour racine <i>mat_pas_mine</i> et pour enfants toutes les combinaisons de possibilités d'état ('#' ou 'D') pour les autres cases non découvertes dans <i>alentours</i> .

ratio_possible_min e	float	Réel entre 0 et 1	Rapport du nombre de feuilles de <i>a_mine</i> correspondant à des situations possibles sur le nombre total de feuilles de <i>a_mine</i> .
ratio_possible_pas_mine	float	Réel entre 0 et 1	Rapport du nombre de feuilles de <i>a_pas_mine</i> correspondant à des situations possibles sur le nombre total de feuilles de <i>a_pas_mine</i> .

III. Stratégie

III.1. Conception du jeu

La fonction la plus importante concernant la stratégie de jeu est la fonction jouer. En effet c'est là que tout est mis en place. Selon le clic utilisé par le joueur et où il a été effectué, la fonction jouer va déterminer l'action réelle à mettre en place à l'aide des matrices.

-Si le joueur fait un clic gauche (action=1) :

-Si la case cliquée comporte un drapeau, alors l'action est impossible.

-Si la case cliquée est en fait une mine, le joueur perd.

-Sinon (la case cliquée est un chiffre) :

-Si la case cliquée est un chiffre différent de 0, on regarde si le chiffre de la case est égal au nombre de drapeaux adjacents à la case :

-S'il manque des drapeaux, on ne fait rien.

-Sinon on dévoile les cases adjacentes à la case cliquée qui n'ont pas de drapeaux, cela peut être une mine si les drapeaux n'étaient pas bien placés.

-Sinon, la case cliquée est un 0, on dévoile tous les 0 sans drapeau autour et les cases adjacentes à ces 0.

-Si le joueur effectue un clic droit (action=2), il posera un drapeau sur la case cliquée si celle-ci n'est pas déjà découverte et ne comporte pas déjà un drapeau. Si le clic est sur une case avec un drapeau, ce dernier est enlevé.

-Si le joueur effectue un clic molette (action=3), l'Intelligence Artificielle est appelée (cf parties (II.2 et III.2)).

III.2. Intelligence Artificielle

Le fonctionnement de notre Intelligence Artificielle peut se découper en 4 grandes étapes.

III.2.i. Détermination de la précision

La taille des arbres manipulés va dépendre du nombre de cases non découvertes présentes dans la matrice *alentours*, qui est la base de notre recherche. C'est pourquoi, la taille de cette matrice, *TAILLE_ELT*, est déterminée en fonction du nombre de cases non découvertes autour de (x_depart, y_depart) = case choisie pour être traitée par l'Intelligence Artificielle. On parcourt les cases encerclant la case (x_depart, y_depart) dans un rayon de plus en plus grand en retenant le nombre de cases non découvertes rencontrées jusqu'alors. Lorsque ce nombre dépasse 8, on arrête le traitement et on initialise la constante *MILIEU* comme étant le rayon actuel – 1. On aura donc $TAILLE_ELT = (2 * MILIEU) + 1$.

Le chiffre 8 a été sélectionné suite à un jeu de tests de notre part comme chiffre le plus pertinent en terme de compromis temps / pertinence du résultat renvoyé par l'Intelligence Artificielle.

III.2.ii. Construction des arbres

Une fois la taille des matrices manipulées choisie, on commence par construire les matrices *alentours*, *mat_mine* et *mat_pas_mine* (cf II.2). À partir de là, on crée les arbres *a_mine* et *a_pas_mine* (cf II.2). Les feuilles de *a_mine* (resp. *a_pas_mine*) représentent tous les états du plateau pouvant découler de la situation initiale représentée par *mat_mine* (resp. *mat_pas_mine*), que ces situations soient possibles ou non du point de vue des règles du démineur.

III.2.iii. Calcul des ratios

Maintenant que les arbres *a_mine* et *a_pas_mine* sont construits, on compte, pour chacun d'eux, leur nombre de feuilles total et leur nombre de feuilles correspondant à un état du plateau possible. Un état de plateau est possible s'il répond aux règles de base du démineur sur la portion de plateau étudiée (*alentours*). Ces règles de base sont vérifiées par la comparaison de chaque numéro présent sur le plateau étudié à son nombre de drapeaux adjacents.

Plus *TAILLE_ELT* est important, plus le résultat renvoyé par l'Intelligence Artificielle est précis.

III.2.iv. Analyse des données

Si *ratio_possible_mine* (resp. *ratio_possible_pas_mine*) est égal à 0, alors il est certain que cette case est sécurisée (resp. non sécurisée) car aucune des situations pour lesquelles l'état initial était une mine (resp. pas de mine) en (x_depart, y_depart) ne mène à un scénario possible.

Si *ratio_possible_mine* (resp. *ratio_possible_pas_mine*) est égal à 1, alors il est certain que cette case est non sécurisée (resp. sécurisée) car toutes les situations pour lesquelles l'état initial était une mine (resp. pas de mine) en (x_depart, y_depart) mènent à un scénario possible.

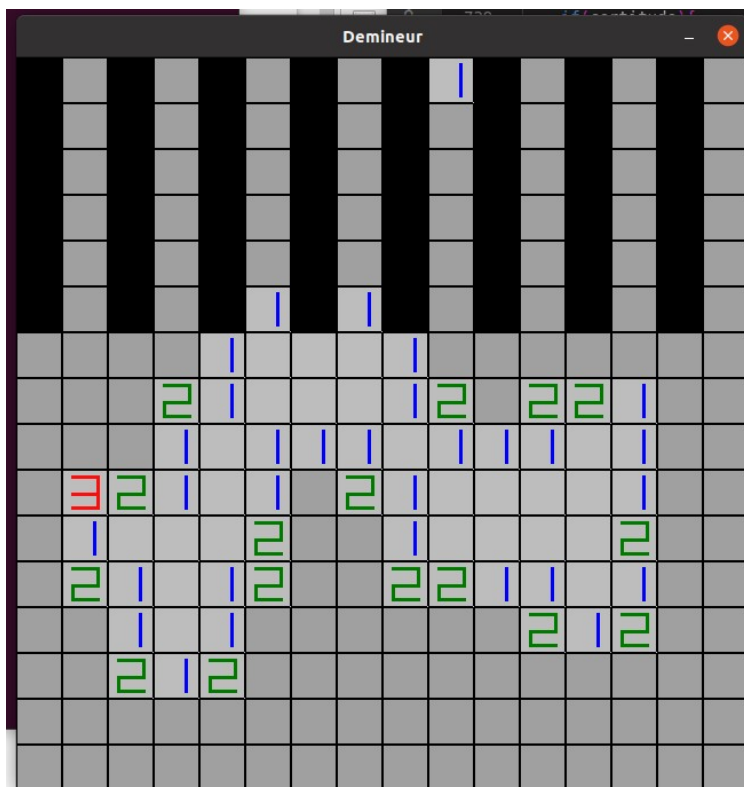
Enfin, si aucun des ratios n'est égal à 0 ou 1, on prend le ratio le plus élevé et on en tire la conclusion adaptée sur l'état de sécurité de la case, en retenant que ce résultat n'est pas certain.

IV. Pour aller plus loin

IV.1. Problèmes repérés

Nous avons rencontré divers problèmes durant la conception du démineur.

Concernant l'affichage graphique : l'affichage graphique apparaît comme souhaité si le code est exécuté à partir de WSL. Cependant, sur une machine fonctionnant sous Ubuntu, l'interface graphique s'affiche mal. Tous les pixels ne s'affichent pas toujours dans la couleur souhaitée, mais ce défaut d'affichage semble se stabiliser lorsque l'on clique plusieurs fois sur une case.



Concernant les fuites mémoire : en compilant avec le flag `-fsanitize=address`, à la fin de l'exécution du programme, nous voyons apparaître un message d'erreur indiquant un oubli de libération de mémoire. Après quelques recherches, nous sommes arrivés à la conclusion que cette erreur était inhérente à la bibliothèque SDL2 utilisée (nous avons la même erreur pour un programme simple n'utilisant que la bibliothèque SDL2).

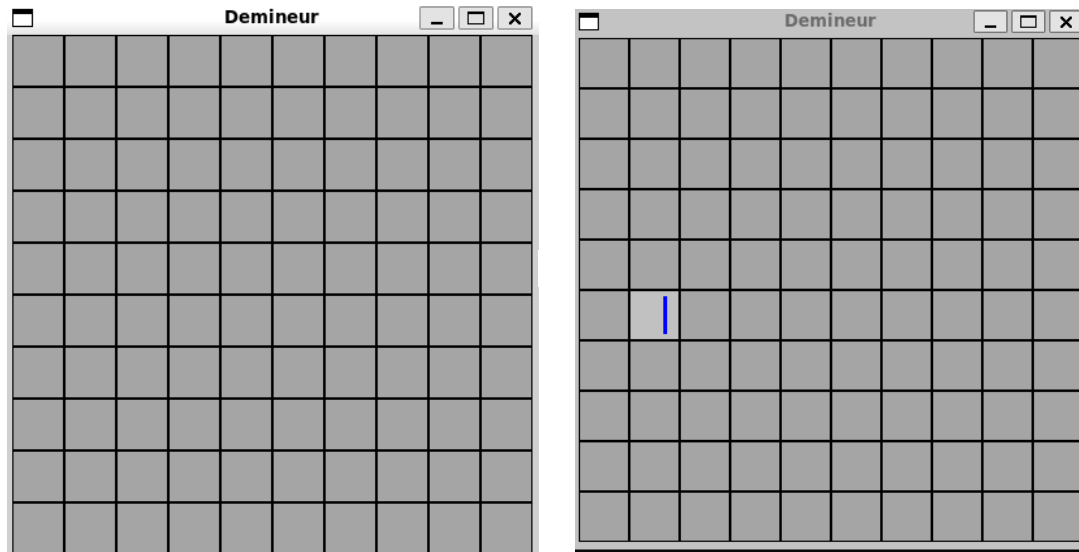
```
==93417==ERROR: LeakSanitizer: detected memory leaks

Direct leak of 17 byte(s) in 1 object(s) allocated from:
    #0 0x7fdf84000808 in __interceptor_malloc ../../../../src/libsanitizer/asan/asan_malloc_linux.cc:144
    #1 0x7fdf82e6e896 in _XlcDefaultMapModifiers (/lib/x86_64-linux-gnu/libX11.so.6+0x5b896)

SUMMARY: AddressSanitizer: 17 byte(s) leaked in 1 allocation(s).
```

Concernant le temps de réponse de l'IA : Si le joueur essaye d'utiliser l'IA au début de la partie, quand aucune case n'est découverte, ou très peu de case découvertes, l'IA prend beaucoup de temps pour renvoyer une réponse, il faut donc utiliser l'IA dans des cas réellement pertinents.

Exemples de cas non pertinents :



Concernant l'écoulement du temps : nous avons créé une fonction permettant de donner le temps depuis le début de la partie en secondes. Cependant, ce temps n'est indiqué qu'après une action, il n'est donc pas affiché en temps réel, pendant que le joueur réfléchit.

IV.2. Pistes d'amélioration

Concernant l'affichage graphique : il doit exister des options dans la bibliothèque SDL2 pour régler le problème d'affichage, sûrement dans la vitesse de rafraîchissement de l'image.

Concernant les fuites mémoire : il faut que nous nous renseignions d'avantage sur la bibliothèque SDL2 et comprendre exactement d'où vient ce problème de mémoire.

Concernant le temps de réponse de l'IA : il faudrait rajouter d'autres tests pour mieux définir quand un cas est pertinent ou non pour ne pas faire 2^8 opérations inutiles.

Concernant l'écoulement du temps : afficher le temps directement dans l'interface graphique serait sûrement plus intéressant et permettrait de l'afficher en temps réel, mais cela imposerait un rafraîchissement de la fenêtre en continue.

Conclusion

Le jeu du Démineur est fonctionnel, en effet les fonctionnalités de base représentant le jeu peuvent être effectuées sans rencontrer de problème. Le temps est indiqué à la fin de la partie pour donner un score au joueur. L'affichage graphique permet, de plus, une meilleure prise en main pour le joueur.

L'Intelligence Artificielle, en calculant un bon nombre de scénarios possibles et impossibles, apporte une aide fiable au joueur dans sa recherche de bombe. Elle ne peut évidemment pas tricher et ne permet donc pas de gagner à 100 % selon les cas rencontrés.

Sources

Différentes ressources ont été utiles pour l'utilisation de la bibliothèque SDL2, son interface graphique et sa gestion d'événements :

[Comment gérer les différents événements en SDL 2](#)

[SDL2/FrontPage - SDL Wiki](#)

[Utiliser la SDL en langage C • Bibliothèque • Zeste de Savoir](#)

Les bibliothèques ARBRE et ELT ont été reprises et adaptées à partir des bibliothèques du même nom fournies en TP.